Python Fundamentals CheatSheet

Table of Contents

1. **Introduction to Python**
2. **Basic Concepts of Python**
3. **Variables and Data Types**
    - Variables
    - Data Types: Integer, Float, String, Boolean
    - Typecasting
4. **Operators**
    - Arithmetic Operators
    - Comparison Operators
    - Logical Operators
5. **Conditional Statements**
    - If-Else Statements
    - If-Elif-Else Statements
6. **Loops in Python**
    - For Loops
    - While Loops
    - Loop Control Statements
    - Nested Loops
7. **Console Input and Type Casting**
    - Using `input()`
    - Type Casting in Python

8. **Functions in Python**
    - Defining a Function
    - Calling a Function
    - Parameters and Arguments
    - Return Values
    - Built-in Functions
    - Default Parameter Values
    - Functions with Multiple Return Values
    - Lambda Functions
    - Function Scope
9. **Lists**
    - Creating a List
    - List Indexing
    - Iterating over a List
    - List Concatenation & Repetition
    - Nested Lists
    - Negative Indexing
    - List Operations
10. **Strings**
    - String Operations
    - Common String Methods
    - String Formatting
11. **Tuples**

- Creating Tuples
- Accessing Elements
- Tuple Methods
- List vs. Tuples

12. **Sets**
    - Creating a Set
    - Set Characteristics
    - Set Operations
    - Set Methods
13. **Dictionaries in Python**
    - Characteristics of Dictionaries
    - Dictionary Methods
    - Iterating over a Dictionary
    - Nested Dictionaries
14. **Python Functional Programming**
    - Lambda Functions
    - `map()`
    - `filter()`
    - `reduce()`
15. **Scope & Function Parameters**
    - Local Scope
    - Enclosing Scope
    - Global Scope

    - Built-in Scope
    - Call By Value vs. Call By Reference
16. **Object-Oriented Programming in Python**
    - Classes and Objects
    - Constructors
    - Abstraction
    - Encapsulation
    - Inheritance
    - Polymorphism
17. **JSON in Python**
18. **Decorators in Python**
19. **Web Scraping**
20. **Best Practices for Writing Python Code**
21. **Additional Resources**
22. **Conclusion**

What is Python?

Python is a high-level, interpreted programming language known for its simplicity and readability. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming.

How to Run a Python Program?

Step 1: Install Python

- Download the latest version of Python from the official website (https://www.python.org/downloads/)
- After that, follow the installation instructions for your operating system.

Step 2: Set up an IDE (e.g., PyCharm Community Edition)

- Download PyCharm Community Edition (https://www.jetbrains.com/pycharm/download/)

- Install and configure PyCharm according to your preferences.
- Create a new Python project and run your Python code within the IDE.

Basic Concepts of Python

1. Python syntax

- Simple, readable, and consistent
- Indentation used for code blocks

2. Using print() function

- Displays text, numbers, or results of expressions
- Automatically adds a newline character

3. Comments

- Single-line comments: start with #
- Multi-line comments: enclosed between triple quotes ("""...""")

```
# This is single line comment

"""
This is multi-line comment
"""

print("Hello there!!!")
```

4. Variables

- They store values for later use.
- No need to declare data type in advance.

5. Data Types

- **Common Types: int, float, str, bool.**

a. Integer

- Whole numbers, positive or negative
- Example: a = 10

b. Float

- Decimal numbers, positive or negative
- Example: b = 6.2

c. String

- Sequence of characters enclosed in quotes (single or double)
- Example: c = "Matt"

d. Boolean

- Represents True or False values
- Example: d = True

```
a = 10          # integer
b = 6.2         # float
c = "Matt"      # string
d = True        # boolean
(True/False)
```

6. Typecasting

After understanding the various data types in Python, it's essential to know how to convert these data types into one another. This process is known as **Typecasting** or **Type Conversion**.

Typecasting allows us to handle various data types seamlessly, ensuring data consistency and enabling mixed-type operations.

Why Use Typecasting?

Typecasting is crucial in scenarios such as:

- Avoiding type errors during operations between different types.
- Parsing user input where the data is typically read as strings.
- Converting between numeric and string data for formatted output.

Types of Typecasting in Python

Python supports two kinds of typecasting:

1. **Implicit Typecasting** – Automatic conversion by Python.
2. **Explicit Typecasting** – Manual conversion using built-in functions.

1. Implicit Typecasting

Implicit typecasting, also known as **type coercion**, is when Python automatically converts one data type to another without explicit instructions from the programmer. This usually occurs when combining different types in operations.

Examples:

1. Adding Integer and Float:

```
integer_value = 5
float_value = 2.0
result = integer_value + float_value  #
Result is 7.0 (float)
```

2. Multiplication Between Integer and Complex:

```
integer_value = 3
complex_value = 4 + 5j
result = integer_value * complex_value  #
Result is 12 + 15j (complex)
```

Explanation: Here, Python automatically converts the integer into a float or complex number during the operation.

2. Explicit Typecasting

Explicit typecasting requires the programmer to manually convert one data type into another using Python's built-in functions. This is necessary when automatic conversion does not occur or when a specific type is required.

Common Functions for Explicit Typecasting:

- `int()`: Converts a value to an integer.
- `float()`: Converts a value to a floating-point number.
- `str()`: Converts a value to a string.
- `list()`, `tuple()`: Converts an iterable to a list or tuple.

Example:

```
age = "25"
int_age = int(age)
# Converts the string "25" into an integer 25

value = 42
str_value = str(value)
# Converts the integer 42 into a string "42"
```

Common Use Cases of Typecasting in Python

- **Reading Numeric Input from the User:** Inputs from the user are typically read as strings. Typecasting is used to convert them into appropriate types for further processing. python
- **Performing Calculations with Mixed Data Types:** Avoiding type errors when performing mathematical operations between different data types.

Key Takeaways

- **Implicit Typecasting** is automatic and occurs during operations involving mixed data types.
- **Explicit Typecasting** is performed manually using Python's built-in functions like `int()`, `float()`, `str()`, etc.
- Typecasting ensures data consistency and prevents errors, especially during mixed-type operations.

7. Operators

a. Arithmetic: +, -, *, /, %.

Operator	Operation	Example
`+`	Addition	`5 + 2 = 7`
`-`	Subtraction	`4 - 2 = 2`
`*`	Multiplication	`2 * 3 = 6`
`/`	Division	`4 / 2 = 2`
`//`	Floor Division	`10 // 3 = 3`
`%`	Modulo	`5 % 2 = 1`
`**`	Power	`4 ** 2 = 16`

```
x = 10
y = 3

# Arithmetic operators
print("x + y =", x + y)
print("x - y =", x - y)

# Comparison operators
print("x == y?", x == y)
print("x < y?", x < y)
```

b. Comparison: ==, !=, >, <, >=, <=.

Operator	Description	Syntax
>	Greater than: True if the left operand is greater than the right	x > y
<	Less than: True if the left operand is less than the right	x < y
==	Equal to: True if both operands are equal	x == y
!=	Not equal to – True if operands are not equal	x != y
>=	Greater than or equal to True if the left operand is greater than or equal to the right	x >= y
<=	Less than or equal to True if the left operand is less than or equal to the right	x <= y
is	x is the same as y	x is y
is not	x is not the same as y	x is not y

```
x = 10
y = 20
name = "Alice"

# Logical operators
print("x > 0 and y > 0:", x > 0 and y > 0)
print("not x < 0:", not x < 0)

# Other operators
print("'A' in name:", 'A' in name)
print("x is y:", x is y)
```

c. Logical: and, or, not.

Operator	Description	Syntax
and	Logical AND: True if both the operands are true	x and y
or	Logical OR: True if either of the operands is true	x or y
not	Logical NOT: True if the operand is false	not x

Conditional Statements

Conditional statements allow your program to make decisions and execute different blocks of code based on certain conditions. This is essential for creating dynamic and responsive programs.

If-Else Statements

The `if-else` statement is used to run a block of code if a specific condition is true. If the condition is false, the `else` block is executed. It's a simple way to handle binary decisions in code.

Syntax:

- Execute different blocks of code based on conditions.

```
if condition:
    # Code block to execute if the condition is
true
else:
    # Code block to execute if the condition is
false
```

Example:

```
num = 10
if num > 0:
    print("Positive number")
else:
    print("Non-positive number")
```

Explanation: Here, if `num` is greater than `0`, "Positive number" is printed. Otherwise, "Non-positive number" is printed.

If-Elif-Else Statements

The `if-elif-else` structure is useful when you need to check multiple conditions sequentially. The program evaluates each `if` or `elif` condition until one is `True`. If none are true, the `else` block executes.

Syntax:

- Execute different blocks of code based on conditions.

```
if condition1:
    # Code block if condition1 is true
elif condition2:
    # Code block if condition2 is true
else:
    # Code block if none of the above conditions
are true
```

Example:

```
num = 0
if num > 0:
    print("Positive number")
elif num == 0:
    print("Zero")
else:
    print("Negative number")
```

Explanation: This code checks if `num` is positive, zero, or negative and prints the appropriate message.

Loops in Python

Loops allow for the repeated execution of a block of code, either over a sequence of items or while a condition is true. They are a fundamental part of controlling the flow of your program.

For Loops

A "`for`" loop is used to iterate over a sequence (such as a list, tuple, dictionary, set, or string) and execute a block of code for each item in that sequence.

Syntax:

- **Execute different blocks of code based on conditions.**

```
for variable in sequence:
    # Code block to execute
```

Example:

```
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)
```

Explanation: This loop iterates over the list of fruits and prints each fruit one by one.

While Loops

The **“`while`”** loop runs as long as a specified condition is true. It's often used when you don't know beforehand how many times you need to loop.

Syntax:

- **Execute different blocks of code based on conditions.**

```
while condition:
    # Code block to execute
```

Example:

```
num = 5
while num > 0:
    print(num)
    num -= 1
```

Explanation: This loop prints the numbers from `5` to `1`. The condition `num > 0` ensures that the loop continues until `num` becomes `0`.

Loop Control Statements

These statements are used to control the flow of loops:

- **`break`:** Exits the loop prematurely.
- **`continue`:** Skips the current iteration and moves to the next one.

Example 1:

```
# Using break
for i in range(5):
    if i == 3:
        break
    print(i)
```

Explanation: This loop prints numbers from `0` to `2` and stops when `i` equals `3`.

Example 2:

```
# Using continue
for i in range(5):
    if i == 3:
        continue
    print(i)
```

Explanation: This loop skips the number `3` and prints the other numbers from `0` to `4`.

Nested Loops

- A nested loop is a loop inside another loop. The inner loop is executed for each iteration of the outer loop.

Syntax:

```
for i in range(3):  # Outer loop
    for j in range(2):  # Inner loop
        # Code block to execute
```

Example:

```
for i in range(3):
    for j in range(2):
        print(f"i = {i}, j = {j}")
```

Explanation: The outer loop runs three times (`i` from `0` to `2`), and for each iteration of the outer loop, the inner loop runs twice (`j` from `0` to `1`).

Additional Example: Nested `while` loops

```python
i = 1
while i <= 3:
    j = 1
    while j <= 2:
        print(f"i = {i}, j = {j}")
        j += 1
    i += 1
```

Explanation: This code uses nested `while` loops to print combinations of `i` and `j` values.

Console Input - input()

Used to read input from the user

```python
# Example of Console Input
user_input = input("Enter your name: ")
print("Hello, " + user_input + "!")
```

Functions / Methods in Python

Functions are reusable blocks of code that perform a specific task. They help make programs more modular, readable, and manageable by allowing you to encapsulate code logic within a single, callable unit. In Python, functions can take inputs (called parameters) and return outputs (called return values).

1. **Defining a Function**
   - A function is defined using the `def` keyword followed by the function name, parentheses containing parameters (if any), and a colon. The body of the function is indented below the definition.

Syntax:

```
def function_name(parameters):
    # Code block (function body)
    return output  # Optional
```

Example:

```
def greet(name):
    print(f"Hello, {name}!")
```

Explanation: This function, `greet`, takes a single parameter `name` and prints a greeting message.

```
def function_name(arguments):
    # function body

    return
```

Here,

- `def` - keyword used to declare a function
- `function_name` - any name given to the function
- `arguments` - any value passed to function
- `return` (optional) - returns value from a function

Let's see an example,

```
def greet():
    print('Hello World!')
```

2. Calling a Function

A function is called by using its name followed by parentheses containing arguments (if any).

Example:

```
greet("Alice")
```

Output:

```
Hello, Alice!
```

3. Parameters and Arguments

- **Parameters:** Variables listed inside the parentheses in the function definition.
- **Arguments:** Values passed to the function when it is called.

Example:

```
def add_numbers(a, b):  # 'a' and 'b' are
parameters
    return a + b

result = add_numbers(5, 3)  # '5' and '3' are
arguments
print(result)

#Output = 8
```

4. Return Values

A function can return a result using the `return` statement. If no `return` statement is used, the function returns `None`.

Example:

```
def square(num):
    return num * num

result = square(4)
print(result)
#Output = 16
```

5. Built-in Functions

Python provides many built-in functions that can be used directly. Some common ones include:

- `len()`: Returns the length of an object.
- `type()`: Returns the type of an object.
- `print()`: Prints output to the console.

Example:

```
print(len("Python"))  # Output: 6
```

6. Default Parameter Values

Functions can have parameters with default values. If no argument is passed for such parameters during the function call, the default value is used.

Syntax:

```
def function_name(param1,
param2=default_value):
    # Code block
```

Example:

```
def greet(name, message="Hello"):
    print(f"{message}, {name}!")

greet("Alice")            # Uses default message
greet("Bob", "Welcome")   # Uses custom message
```

Output:

```
Hello, Alice!
Welcome, Bob!
```

7. Function with Multiple Return Values

A function can return multiple values as a tuple.

Example:

```python
def calculate(a, b):
    sum = a + b
    difference = a - b
    return sum, difference

result = calculate(10, 5)
print(result)

# Output: (15, 5)
```

8. Lambda Functions (Anonymous Functions)

Lambda functions are small, anonymous functions defined using the `lambda` keyword. They can take any number of arguments but have only one expression.

Syntax:

```
lambda arguments: expression
```

Example:

```python
square = lambda x: x * x
print(square(5))

# Output: 25
```

9. Function Scope

The scope of a variable refers to where in the program it can be accessed. Python has 4 levels of variable scope:

1. **Local Scope**: Variables declared inside a function.
2. **Enclosing (Non-local) Scope**: Variables in the enclosing function (for nested functions).
3. **Global Scope**: Variables declared at the top level of a module.
4. **Built-in Scope**: Variables pre-defined in Python (e.g., `print`, `len`).

Example:

```
x = 10  # Global variable

def outer():
    y = 20  # Enclosing variable
    def inner():
        z = 30  # Local variable
        print(x, y, z)
    inner()

outer()
```

Output:

```
10 20 30
```

10. Common Built-in Functions

Function	Description	Example	Output
len()	Returns the length of an object	len("Python")	6
type()	Returns the type of an object	type(42)	<class 'int'>
print()	Prints the specified message	print("Hello World")	Hello World
max()	Returns the largest item	max([1, 2, 3, 4])	4
min()	Returns the smallest item	min([1, 2, 3, 4])	1
sum()	Returns the sum of all items	sum([1, 2, 3, 4])	10

11. Function Recursion

Recursion occurs when a function calls itself. It's commonly used for problems that can be broken down into smaller, similar sub-problems.

Example:

```python
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(5))  # Output: 120
```

Key Takeaways

- Functions help encapsulate code, making it reusable and manageable.
- Parameters are inputs to functions, while arguments are values passed during function calls.
- Use `return` to send back output from a function.
- Default parameter values make function arguments optional.
- Lambda functions provide a quick way to write simple, single-expression functions.

Lists

Lists are used to store multiple items in a single variable. They are versatile data structures that can hold elements of various data types. Lists are ordered, changeable (mutable), and allow duplicate values. List items are indexed, with the first item having an index of `[0]`.

1. Creating List:

```
shopping_list = ['eggs', 'milk', 'bread', 'vegetables']
```

2. List Indexing:

```
first_item = shopping_list[0]  # Accessing the first item
```

3. Iterating over list:

```
my_list = [1, 2, 3, 4, 5]

for element in my_list:
    print(element)
```

4. List Concatenation & Repetition:

```
list1 = [1, 2, 3]
list2 = ['a', 'b', 'c']

concatenated_list = list1 + list2
```

```
original_list = [1, 2, 3]
repeated_list = original_list * 3
```

5. Nested Lists:

```
nested_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

6. Negative Indexing:

```
my_list = [1, 2, 3, 4, 5]

# Access the last element
last_element = my_list[-1]

# Access the second-to-last element
second_last_element = my_list[-2]

print(f"Last element: {last_element}")
print(f"Second-to-last element: {second_last_element}")
# Output:
# Last element: 5
# Second-to-last element: 4
```

7. List Operations:

- **Accessing Elements:** Use list[index] to access an element.

- **Slicing:** Use list[start:end] to access a subset of the list.
- **Modifying Elements:** Assign a new value using list[index] = new_value.
- **Adding Elements:** Use list.append(item) or list.insert(index, item).
- **Appending Elements**

```
# Accessing
print(fruits[0])  # Output: apple

# Slicing
print(fruits[0:2])  # Output: ['apple', 'banana']

# Modifying
fruits[1] = "blueberry"
print(fruits)  # Output: ['apple', 'blueberry', 'cherry']

# Adding
fruits.append("orange")
print(fruits)  # Output: ['apple', 'blueberry', 'cherry',
'orange']

# Removing
fruits.remove("apple")
print(fruits)  # Output: ['blueberry', 'cherry', 'orange']
```

Common List Methods:

Method	Description	Example	Output
append()	Adds an element at the end of the list.	fruits.append("kiwi")	['apple', 'banana', 'kiwi']
insert()	Inserts an element at the specified index.	fruits.insert(1, "mango")	['apple', 'mango', 'banana']
remove()	Removes the first item with the specified value.	fruits.remove("banana")	['apple', 'kiwi']
pop()	Removes the element at the specified index.	fruits.pop(2)	['apple', 'kiwi']
len()	Returns the number of elements in the list.	len(fruits)	3
sort()	Sorts the list in ascending order.	fruits.sort()	['apple', 'banana', 'kiwi']

Strings

Strings are sequences of characters enclosed in single, double, or triple quotes. They are immutable, meaning once created, they cannot be modified.

1. String Operations

- **Concatenation:** Combines two strings using `+`.
- **Indexing:** Accesses a character in a string using its index.
- **Slicing:** Extracts a substring using `string[start:end]`.

Example:

```
name = "Python"
print(name[1])        # Output: y
print(name[0:3])      # Output: Pyt
```

2. Common String Methods

Method	Description	Example	Output
len()	Returns the length of the string.	len("Python")	6
lower()	Converts all characters to lowercase.	"HELLO".lower()	hello
upper()	Converts all characters to uppercase.	"hello".upper()	HELLO
strip()	Removes leading and trailing whitespaces.	" hello ".strip()	hello
replace()	Replaces a substring with another string.	"hello".replace("h", "j")	jello

3. String Formatting

- **Concatenation:** Using `+` to combine strings.
- **f-strings:** Use `f"{variable}"` to format strings.

Example:

```
name = "Alice"
age = 30
print(f"{name} is {age} years old.")
# Output: Alice is 30 years old.
```

Methods of Python String

Besides those mentioned above, there are various string methods present in Python. Here are some of those methods:

Methods	Description
upper()	converts the string to uppercase
lower()	converts the string to lowercase
partition()	returns a tuple
replace()	replaces substring inside
find()	returns the index of first occurrence of substring
rstrip()	removes trailing characters
split()	splits string from left
startswith()	checks if string starts with the specified string
isnumeric()	checks numeric characters
index()	returns index of substring

Tuples

Tuples are ordered, immutable collections that can contain multiple items. They are similar to lists but cannot be modified once created.

1. Tuple Creation

a. Empty Tuple:

```
empty_tuple = ()
```

b. Single Element Tuple:

```
single_element_tuple = (42,)
```

c. Mixing Data Types:

```
mixed_tuple = (1, "hello", 3.14)
```

2. Accessing Elements

a. Indexing:

```
my_tuple = (10, 20, 30)
first_element = my_tuple[0]
```

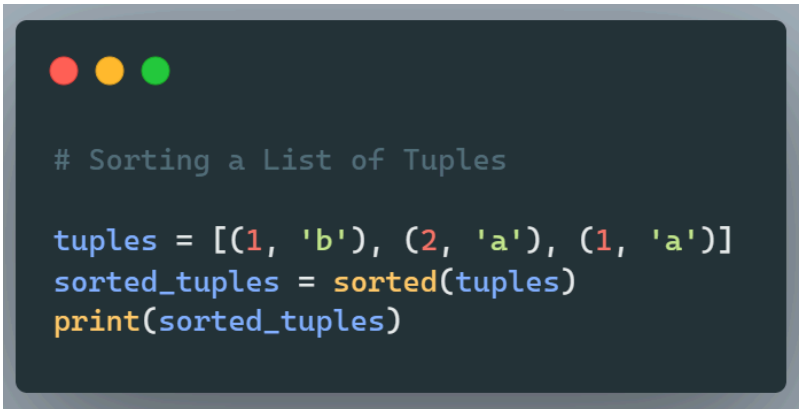
b. Negative Indexing

```
last_element = my_tuple[-1]
```

c. Slicing:

```
my_tuple = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

# Slice from index 2 to 5 (exclusive)
sliced_tuple = my_tuple[2:5]

print(f"Sliced Tuple: {sliced_tuple}")
# Output: Sliced Tuple: (3, 4, 5)
```

d. Sorting



3. Tuple Methods

- len(): Find the length of tuple
- count(): counting occurrences of a value in a tuple
- index(): Finding the index of a value in a tuple

```
my_tuple = (1, 2, 2, 3, 4)
length = len(my_tuple)
count_of_2 = my_tuple.count(2)
index_of_3 = my_tuple.index(3)
```

Method	Description	Example	Output
len()	Returns the number of elements in a tuple.	len(my_tuple)	3
count()	Counts the occurrences of a value.	my_tuple.count(2)	1
index()	Returns the index of a value.	my_tuple.index(3)	2
Method	Description	Example	Output
len()	Returns the number of elements in a tuple.	len(my_tuple)	3
count()	Counts the occurrences of a value.	my_tuple.count(2)	1

List vs Tuples

List	Tuples
Lists are mutable.	Tuples are Immutable.
Use when you need a collection that might change	Use when you have a collection that should remain constant.
Because lists are mutable, they may require more memory & might have a slight performance overhead.	Immutable tuples are generally more memory-efficient and can be faster for certain operations.
my_list = [1, 2, 3]	my_tuple = (1, 2, 3)

Sets

Sets are used to store multiple items in a single variable.

- Set *items* are unchangeable, but you can remove items and add new items.
- Set items are unordered, unchangeable, and do not allow duplicate values.

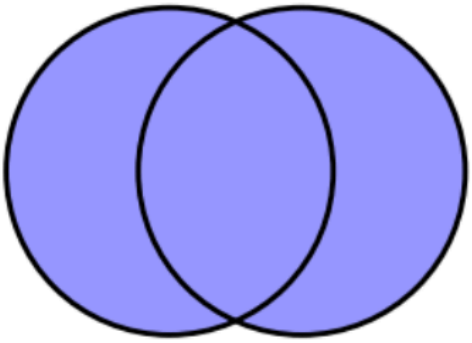
1. **Creating a Set**:

```
thisset = {"apple", "banana", "cherry"}
print(thisset)
```

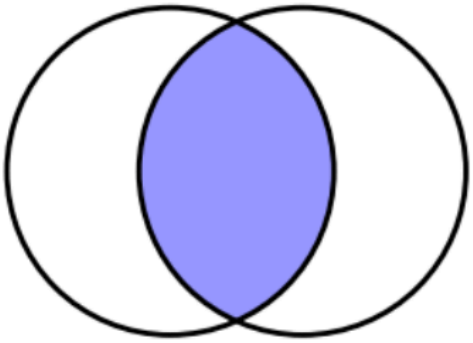
2. **Set Characteristics**:
    - **Unordered**: The items in a set do not have a defined order.
    - **Unchangeable**: While set items themselves cannot be modified, you can add or remove items from the set.
    - **No Duplicates**: Sets automatically remove any duplicate values.

3. Set Operations

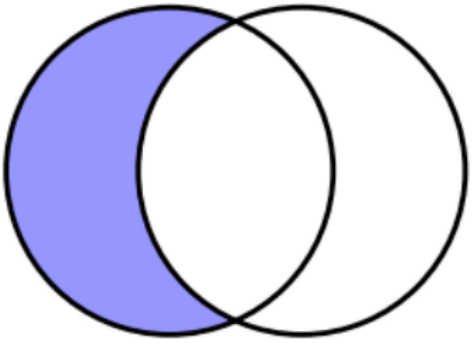
Set Union (S1 ∪ S2)



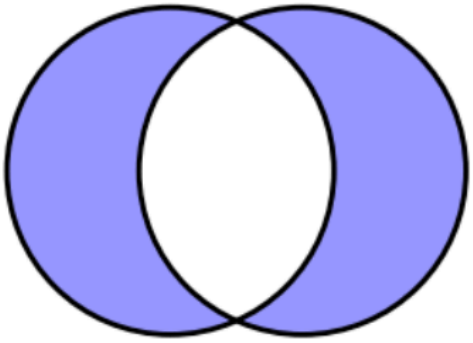
Set Intersection (S1 ∩ S2)



Set Difference (S1 - S2)



Set Symmetric Difference (S1 Δ S2)



a. **Union**:
   - Combines all unique elements from two sets.

```
set1 = {"apple", "banana", "cherry"}
set2 = {"cherry", "date", "elderberry"}
union_set = set1.union(set2)
print(union_set)
# Output: {'apple', 'banana', 'cherry', 'date',
'elderberry'}
```

b. **Intersection**:
   - Finds common elements between two sets.

```
intersection_set = set1.intersection(set2)
print(intersection_set)

# Output: {'cherry'}
```

c. **Difference**:
   - Returns items present in the first set but not in the second.

```
difference_set = set1.difference(set2)
print(difference_set)

# Output: {'apple', 'banana'}
```

d. **Symmetric Difference**:
   - Returns elements that are in either of the sets, but not in both.

```
symmetric_diff_set = set1.symmetric_difference(set2)
print(symmetric_diff_set)

# Output: {'apple', 'banana', 'date', 'elderberry'}
```

3. Set Methods

a. **`clear()`**:
   - Removes all elements from the set.

```
thisset.clear()
print(thisset)

# Output: set()
```

b. **`copy()`**:
   - Returns a copy of the set.

```
new_set = thisset.copy()
print(new_set)

# Output: {'apple', 'banana', 'cherry'}
```

Dictionary in Python

Dictionaries are a built-in data structure in Python that store data in **key-value pairs**. Each key is unique, and it is used to access the associated value. Unlike lists and tuples, dictionaries are unordered but allow for changeable (mutable) elements.

Key Characteristics of Dictionaries:

- **Ordered** (in Python 3.7 and later): Maintains the order of elements based on insertion.
- **Changeable**: You can modify, add, or remove items in a dictionary.
- **No Duplicates**: Each key must be unique; if a duplicate key is assigned, the latest value will overwrite the previous one.

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
```

Dictionary Methods:

1. clear():

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
my_dict.clear()
print(my_dict)  # Output: {}
```

2. copy():

```
original_dict = {'a': 1, 'b': 2}
new_dict = original_dict.copy()
print(new_dict)  # Output: {'a': 1, 'b': 2}
```

3. get(key[,default])

```
my_dict = {'a': 1, 'b': 2}
value = my_dict.get('c', 0)
print(value)  # Output: 0
```

4. pop(key[,default])

```
my_dict = {'a': 1, 'b': 2}
value = my_dict.pop('a')
print(value)  # Output: 1
```

5. update([other])

```
my_dict = {'a': 1, 'b': 2}
my_dict.update({'b': 3, 'c': 4})
print(my_dict)  # Output: {'a': 1, 'b': 3, 'c': 4}
```

6. popitem()

```
my_dict = {'a': 1, 'b': 2}
item = my_dict.popitem()
print(item)  # Output: ('b', 2)
```

7. keys()

```
my_dict = {'a': 1, 'b': 2}
keys = my_dict.keys()
print(keys)  # Output: dict_keys(['a', 'b'])
```

8. values()

```
my_dict = {'a': 1, 'b': 2}
values = my_dict.values()
print(values)  # Output: dict_values([1, 2])
```

9. items()

```
my_dict = {'a': 1, 'b': 2}
items = my_dict.items()
print(items)  # Output: dict_items([('a', 1), ('b', 2)])
```

10. Iterating over dictionary:

- **Iterating over keys:**

```
my_dict = {'a': 1, 'b': 2, 'c': 3}

for value in my_dict.values():
    print(value)
```

- **Iterating over values:**

```
my_dict = {'a': 1, 'b': 2, 'c': 3}

for value in my_dict.values():
    print(value)
```

- **Iterating over key-values pairs:**

```
my_dict = {'a': 1, 'b': 2, 'c': 3}

for key, value in my_dict.items():
    print(key, value)
```

11. Nested Dictionary:

```
family = {
  "child1": {"name": "Alice", "age": 5},
  "child2": {"name": "Bob", "age": 7},
  "child3": {"name": "Charlie", "age": 3}
}

# Accessing nested dictionary elements
print(family["child2"]["name"])  # Output: Bob
```

Python Functional Programming

In Python, lambda, map, filter, and reduce are powerful tools for writing concise and expressive code, especially when working with functions and collections.

1. lambda:

A lambda function is a small anonymous function defined with the lambda keyword. It can take any number of arguments but can only have one expression.

Syntax:

lambda arguments: expression

Example:

A lambda function that adds 10 to the input

```
# A lambda function that adds 10 to the input
add_ten = lambda x: x + 10
print(add_ten(5))  # Output: 15
```

2. Map

The map function applies a given function to all items in an input list (or any iterable) and returns a map object (which is an iterator).

Syntax:

map(function, iterable)

Example:

```
# Using map to square all numbers in a list
numbers = [1, 2, 3, 4, 5]
squared = map(lambda x: x**2, numbers)
print(list(squared))  # Output: [1, 4, 9, 16, 25]
```

3. Filter

The filter function constructs an iterator from elements of an iterable for which a function returns true.

Syntax: filter(function, iterable)

Example:

```
# Using filter to get even numbers from a list
numbers = [1, 2, 3, 4, 5, 6]
evens = filter(lambda x: x % 2 == 0, numbers)
print(list(evens))  # Output: [2, 4, 6]
```

4. **Reduce:**

   The reduce function from the functools module applies a given function to the first two items of an iterable, then applies the same function to the result and the next item, and so on, reducing the iterable to a single cumulative value.

   - **Syntax:**

```
from functools import reduce
reduce(function, iterable)
```

   Example:

```
from functools import reduce

# Using reduce to find the product of all numbers in a list
numbers = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, numbers)
print(product)  # Output: 120
```

Summary

- **lambda**: Creates a small anonymous function.
- **map**: Applies a function to all items in an iterable.
- **filter**: Filters items in an iterable based on a function that returns true or false.
- **reduce**: Reduces an iterable to a single cumulative value by applying a function.

These functions are useful for writing clean, concise, and functional-style code in Python.

Scope & Function Parameters

Scope of variables refers to the region of a program where the variable is accessible. Python has 4 levels of variables scope:

a. Local Scope

```
def my_function():
    # This variable is local to my_function
    local_var = "I am a local variable"
    print("Inside the function:", local_var)

# Calling the function
my_function()
```

b. Enclosing (or non-local) scope

```
def outer_func():
    outer_var = 30

    def inner_func():
        nonlocal outer_var
        print(outer_var)

    inner_func()

outer_func()  # Output: 30
```

c. Global Scope

```
global_var = 10

def func():
    global global_var
    print(global_var)

func()  # Output: 10
```

d. Built in Scope

```
# Using built-in functions
print("Hello, world!")  # Output: Hello, world!
print(len([1, 2, 3]))    # Output: 3
print(abs(-5))           # Output: 5
```

e. Call By Value:

```
def modify_int(x):
    x = x + 1
    print("Inside function:", x)

num = 5
modify_int(num)
print("Outside function:", num)  # Output: Inside function: 6, Outside function: 5
```

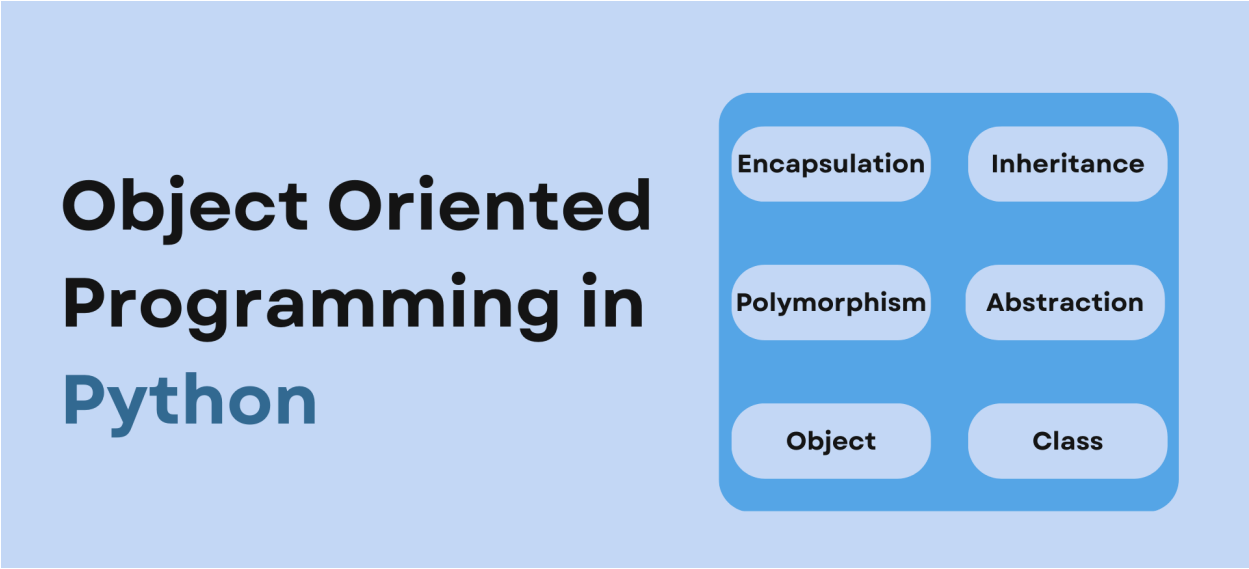
f. Call By Reference:

```python
def modify_list(lst):
    lst.append(4)
    print("Inside function:", lst)

my_list = [1, 2, 3]
modify_list(my_list)
print("Outside function:", my_list)  # Output: Inside function: [1, 2, 3, 4]
```

OOPS in Python

Object Oriented Programming is a programming paradigm that revolves around objects, which are instances of classes.

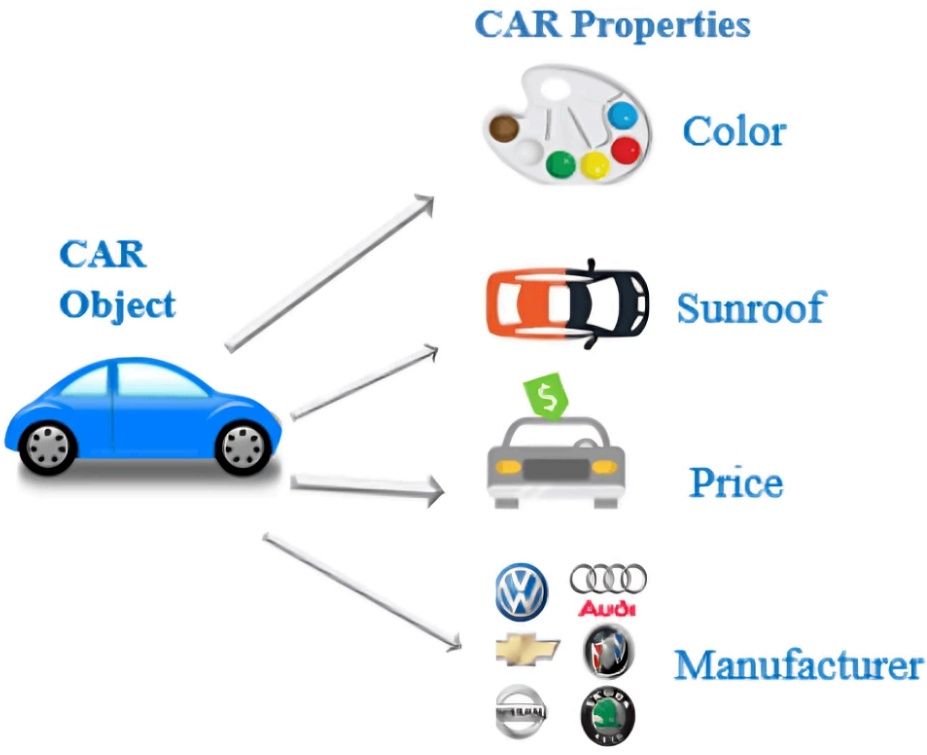
Classes define the structure and behavior of objects , and can encapsulate data and methods.



Classes and Objects:

- **Classes** define the structure and behavior of objects. They encapsulate data (attributes) and methods (functions) that operate on that data.
- **Objects** are instances of classes. When you create an object, you instantiate a class, which means that each object has its own unique set of attributes defined by the class.

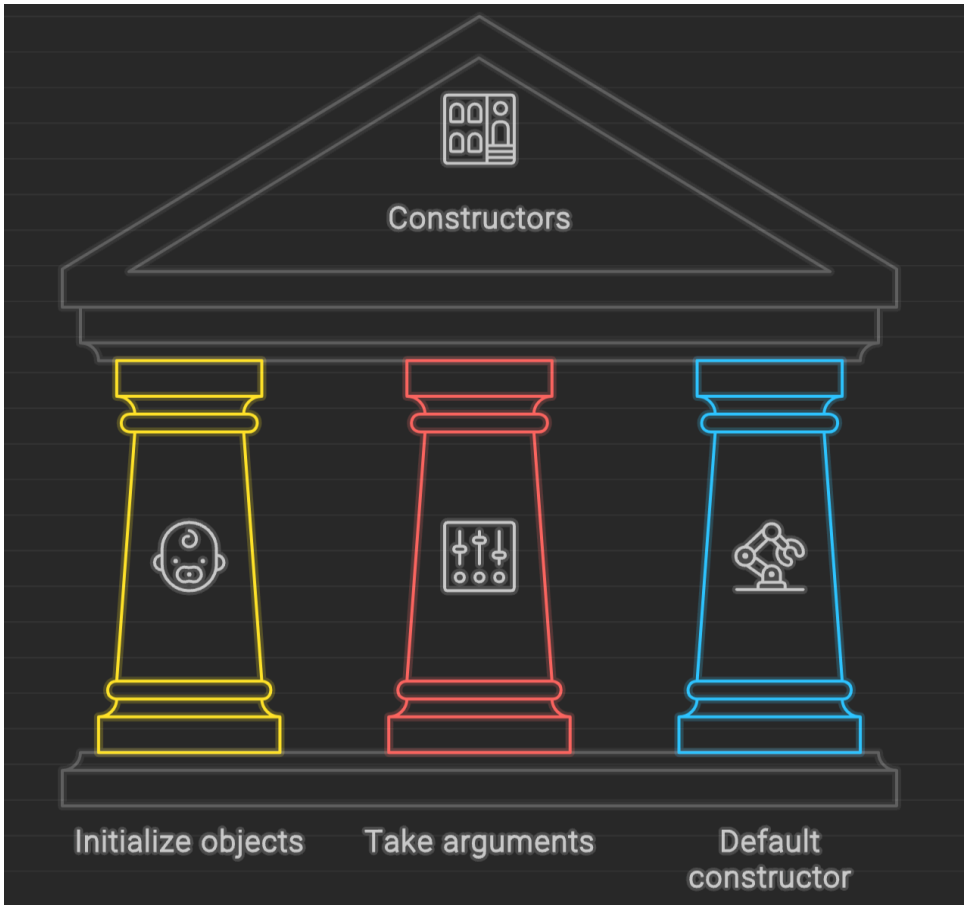
```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

my_dog = Dog("Buddy", "Golden Retriever")
```



Constructors

Constructors are special methods that are automatically called when an object is created. In Python, the `__init__` method is the constructor.

```python
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

my_car = Car("Toyota", "Corolla")
```



Default Constructor

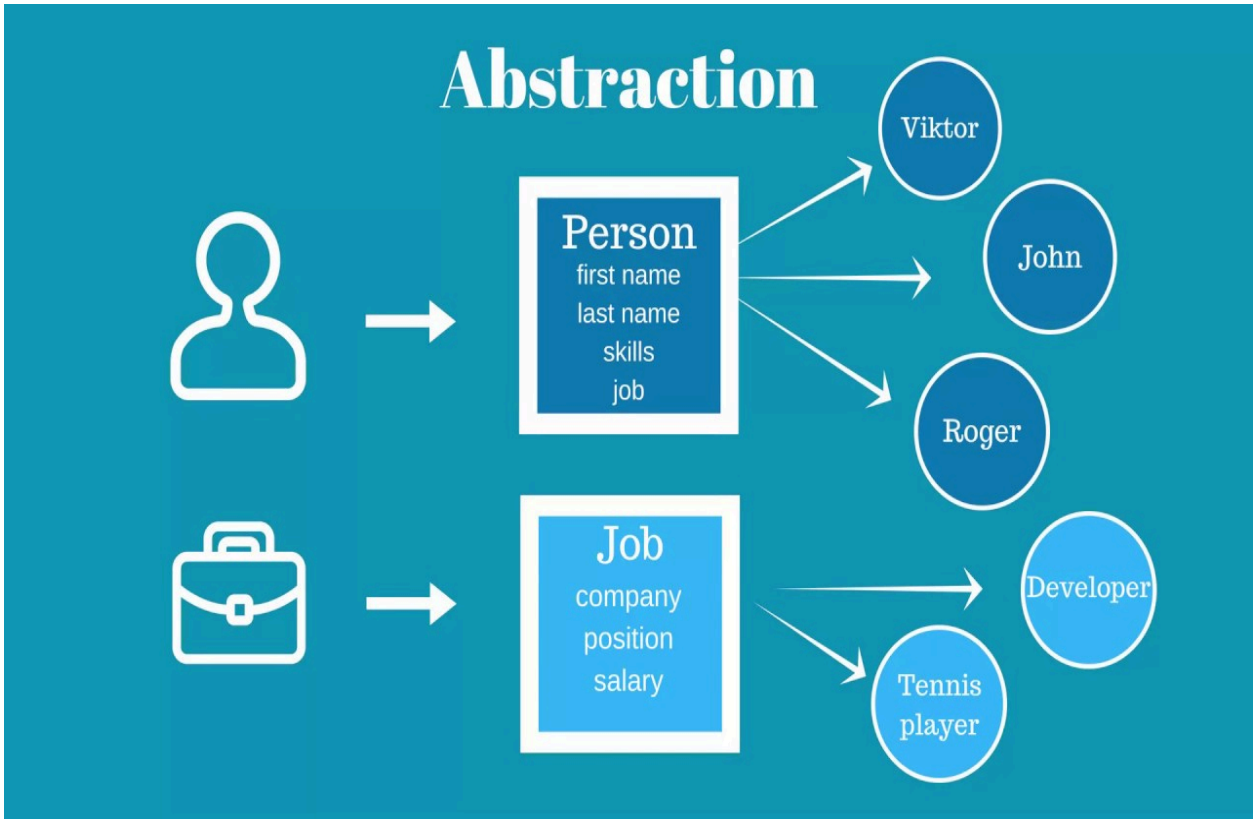
- A **default constructor** is a special constructor used when no other constructor is provided.
- It is automatically called when an object is created.
- The default constructor takes **no arguments**.

Characteristics of Default Constructor:

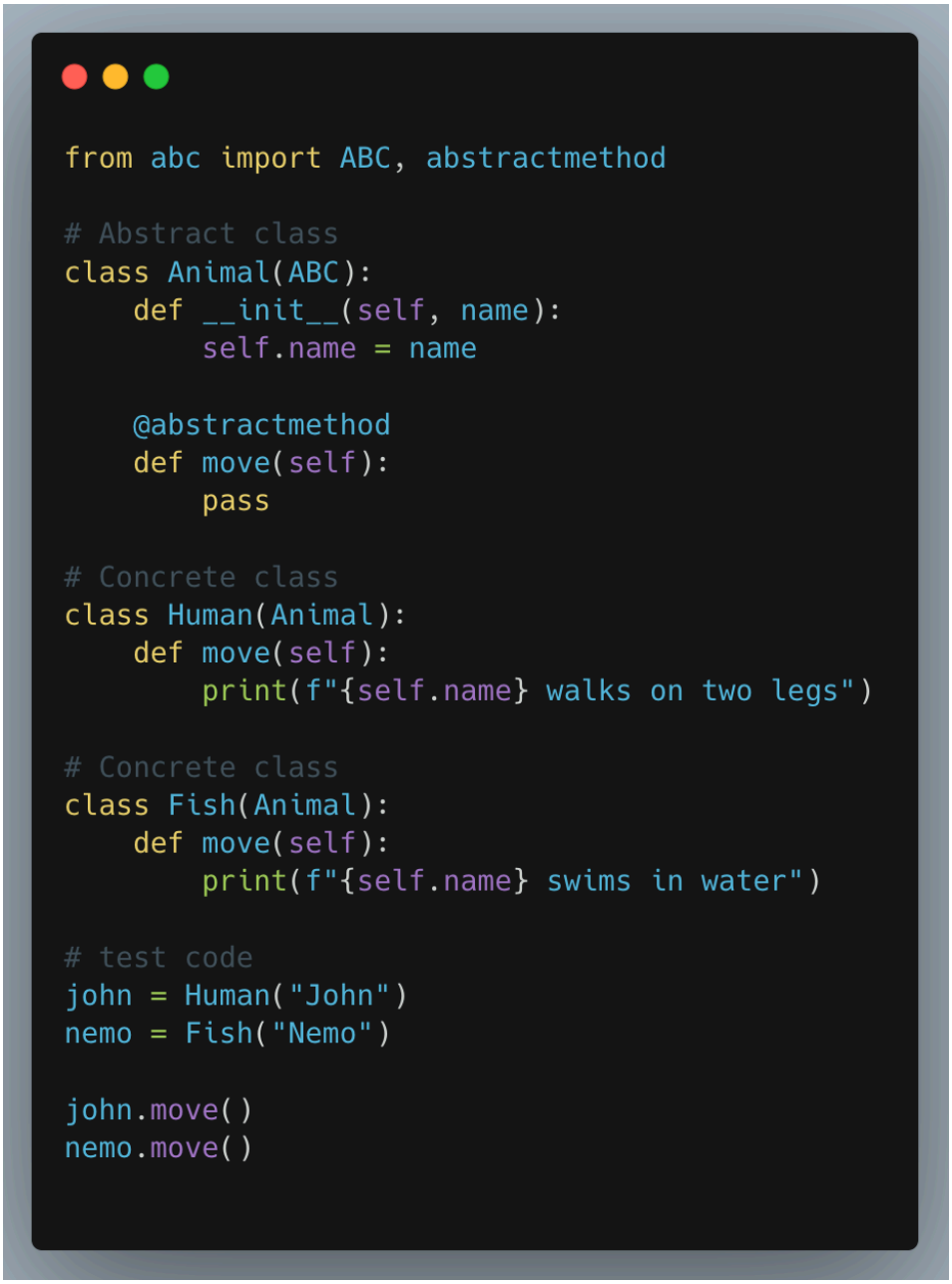
- **Automatic Invocation**: It is automatically invoked during object creation and initialization.
- **No Arguments**: It has empty parentheses and does not accept parameters.

Abstraction

Abstraction is the concept of hiding complex implementation details and showing only the necessary features. It allows programmers to manage complexity by breaking down a system into smaller, manageable parts.



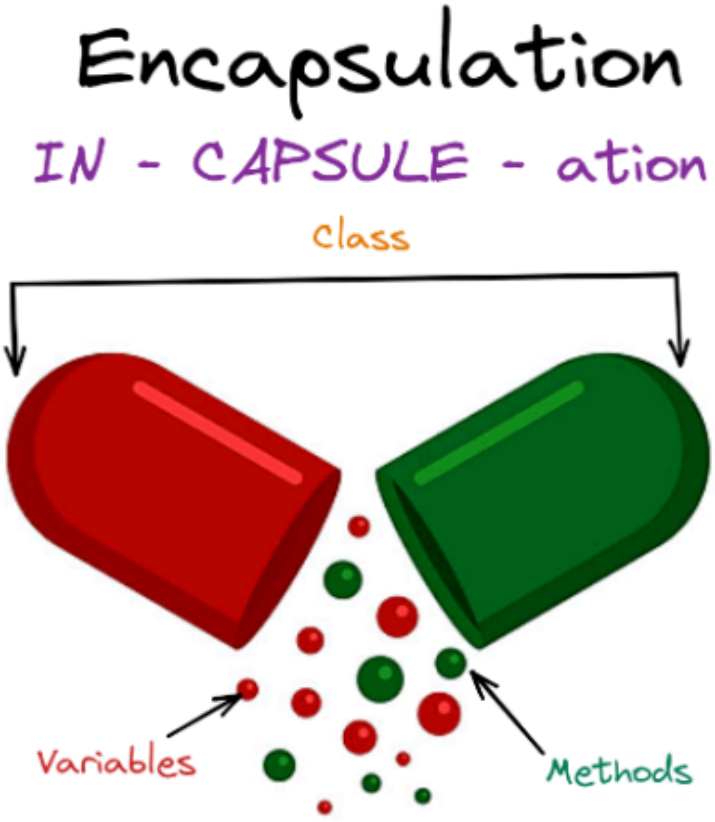
Example of Abstraction



```python
from abc import ABC, abstractmethod

# Abstract class
class Animal(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def move(self):
        pass

# Concrete class
class Human(Animal):
    def move(self):
        print(f"{self.name} walks on two legs")

# Concrete class
class Fish(Animal):
    def move(self):
        print(f"{self.name} swims in water")

# test code
john = Human("John")
nemo = Fish("Nemo")

john.move()
nemo.move()
```

Encapsulation

Encapsulation involves bundling data (attributes) and methods that operate on that data into a single unit, i.e., a class. This keeps the data safe from outside interference and misuse.

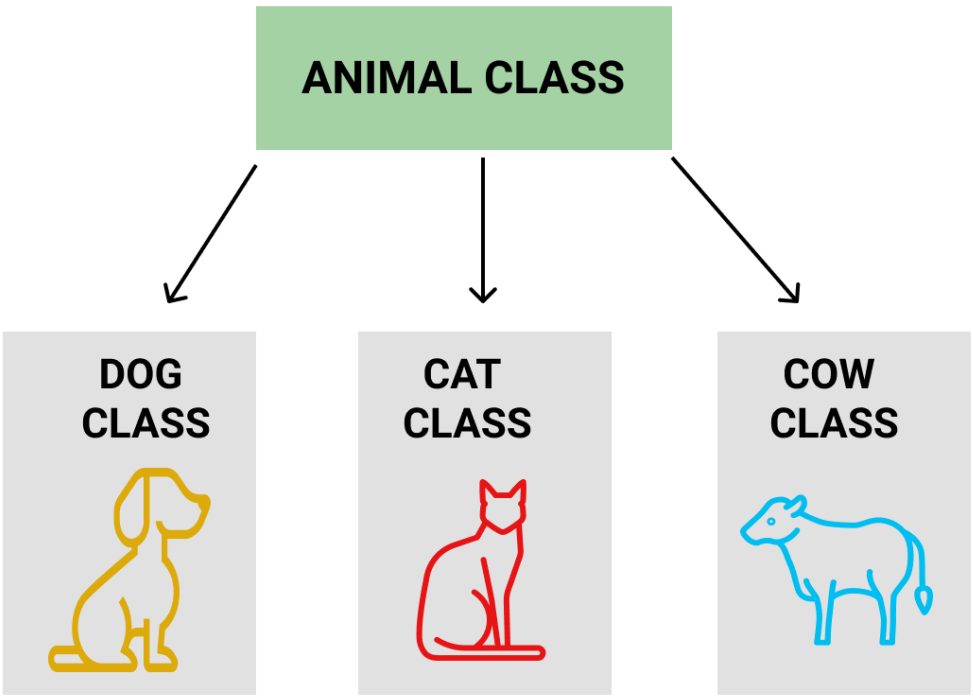


Example:

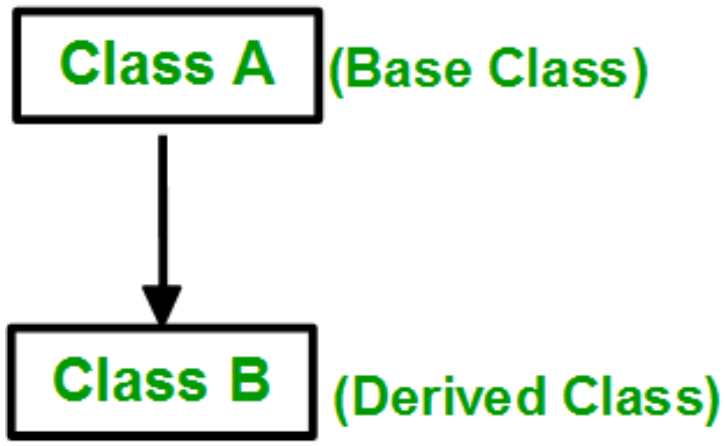
```python
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance  # Private attribute

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance
```

Inheritance

Inheritance allows the creation of a new class (derived class) from an existing class (base class). This promotes code reusability.



1. Single Inheritance:

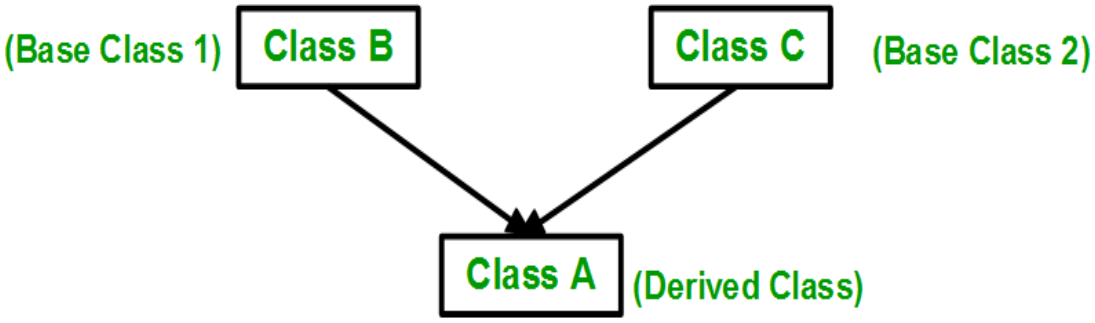


Example:

```
# Parent class
class Bird:
    def __init__(self):
        print("Bird is ready")

    def whoisThis(self):
        print("Bird")

    def swim(self):
        print("Swim faster")

# Child class
class Penguin(Bird):
    def __init__(self):
        print("Penguin is ready")
```

2. Multiple inheritance:

When a class inherits from more than one base class.

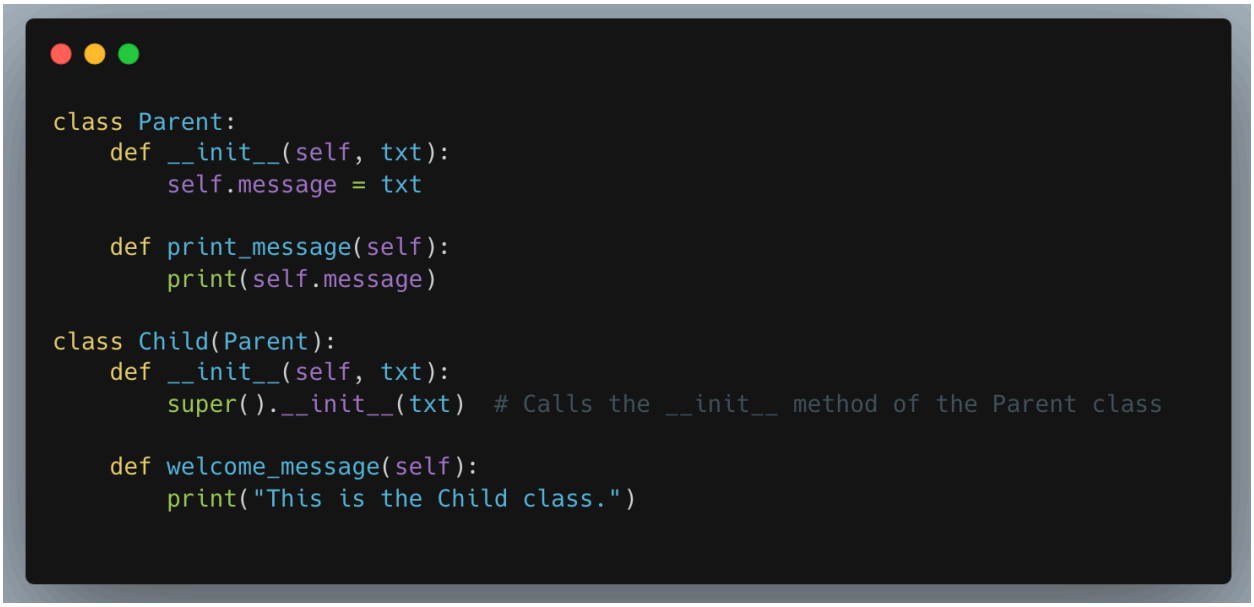


Example:

```python
# Parent class 1
class Bird:
    def __init__(self):
        print("Bird is ready")

    def whoisThis(self):
        print("Bird")

    def swim(self):
        print("Swim faster")

# Parent class 2
class Animal:
    def __init__(self):
        print("Animal is ready")

    def whoisThis(self):
        print("Animal")

    def run(self):
        print("Run faster")

# Child class
class Penguin(Bird, Animal):
    def __init__(self):
        print("Penguin is ready")

b = Penguin()
b.swim()
b.run()
```

Super()

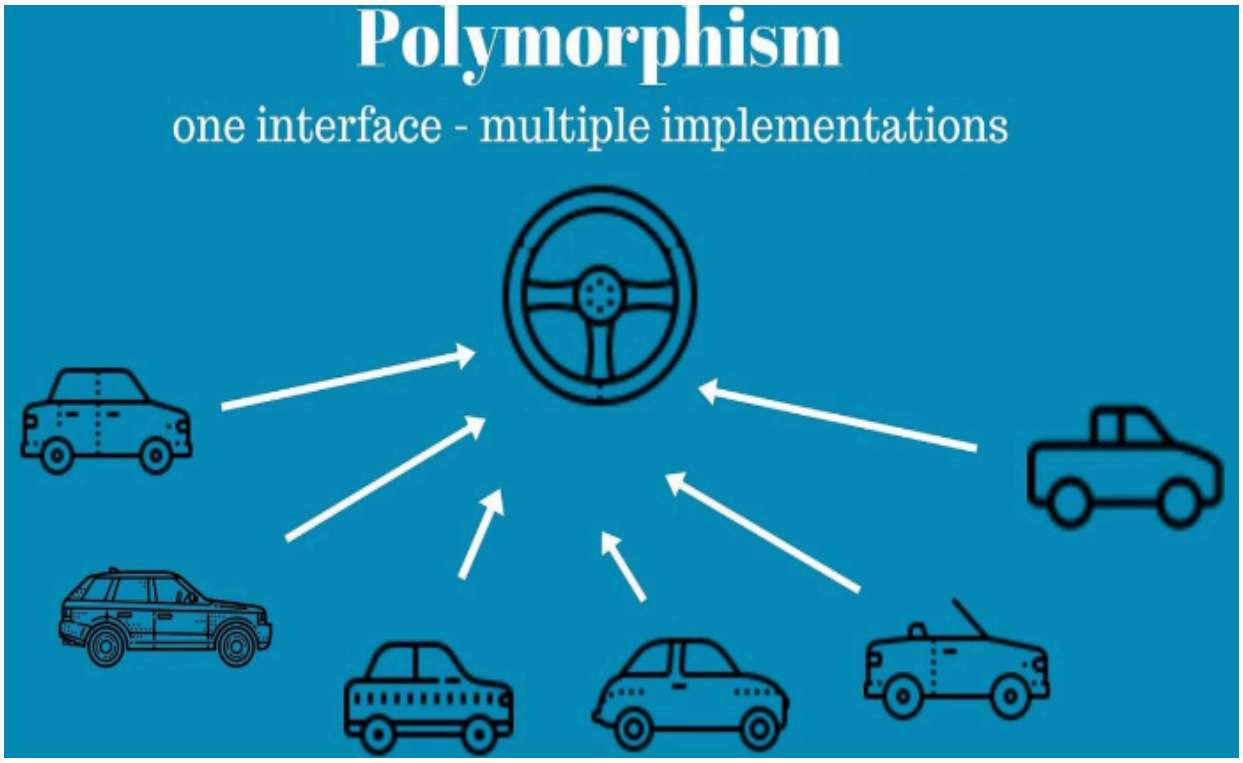
The super keyword is used to refer to the parent class, also known as the superclass.



```python
class Parent:
    def __init__(self, txt):
        self.message = txt

    def print_message(self):
        print(self.message)

class Child(Parent):
    def __init__(self, txt):
        super().__init__(txt)  # Calls the __init__ method of the Parent class

    def welcome_message(self):
        print("This is the Child class.")
```

Polymorphism

Ability of different objects to respond to the same method in different ways



1. **Static Polymorphism**

   In static polymorphism , the method resolution is done at compile time , based on the number and types of arguments passed.

```python
def add(x, y, z=0):
    return x + y + z

print(add(2, 3))      # Output: 5
print(add(2, 3, 4))   # Output: 9
```

2. Dynamic polymorphism

In dynamic polymorphism , the method resolution is done at run time , based on the actual type of object which the method called on.

```python
class Bird:
    def intro(self):
        print("There are many types of birds.")

    def flight(self):
        print("Most of the birds can fly but some cannot.")

class Sparrow(Bird):
    def flight(self):
        print("Sparrows can fly.")

class Ostrich(Bird):
    def flight(self):
        print("Ostriches cannot fly.")
```

JSON in Python

JSON (JavaScript Object Notation) is a lightweight data interchange format.
Python provides a built-in module `json` for serializing (encoding) and deserializing (decoding) JSON data.

a. Serializing (encoding) JSON

```python
import json

data = {
    "firstName": "Jane",
    "lastName": "Doe",
    "hobbies": ["running", "sky diving", "singing"],
    "age": 35,
    "children": [
        {"firstName": "Alice", "age": 6},
        {"firstName": "Bob", "age": 8}
    ]
}

json_string = json.dumps(data, indent=2)  # Indent for readability
print(json_string)
```

b. Deserializing (decoding) JSON

```
json_data = '{"firstName": "Jane", "lastName": "Doe", ...}'
decoded_data = json.loads(json_data)
print(decoded_data["firstName"])  # Access individual values
```

c. Reading from a JSON file

```
# Reading from a JSON file
with open('path_to_file/person.json', 'r') as f:
    data = json.load(f)
print(data)  # Output: {'name': 'Bob', 'languages': ['English', 'French']}
```

d. Writing a JSON file

```
person_dict = {
    "name": "Bob",
    "languages": ["English", "French"],
    "married": True,
    "age": 32
}

with open('person.txt', 'w') as json_file:
    json.dump(person_dict, json_file)
```

Decorators in Python

Decorators are functions that modify or enhance other functions without changing their actual code. They are applied using the `@decorator_name` syntax.

How Decorators Work?

A decorator is applied to a function using the @decorator_name syntax above the function definition.

Example of a Simple Decorator:

```
def my_decorator(func):
    def wrapper():
        print("Something before the function")
        func()
        print("Something after the function")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

Output:

```
Something before the function is called.
Hello!
Something before the function is called.
```

Explanation

- **my_decorator**: This is the decorator function. It takes a function ‘func’ as an argument.
- **wrapper**: This inner function wraps the original function, adding behavior before and after calling func.
- **@my_decorator**: This syntax applies my_decorator to say_hello.

Decorators are commonly used for logging, access control, instrumentation, and other cross-cutting concerns in Python programs.

Web Scraping

Web Scraping is the process of extracting data from websites. Python's `BeautifulSoup` library is commonly used for this purpose.

1. **Installing Beautiful Soup**:

```
pip install beautifulsoup4
```

2. **Parser Types**:
    - Beautiful Soup supports various parsers, like HTML, XML, and lxml.

3. **Functions: Parameters vs. Arguments**
    - **Parameters**: Variables defined in the function's signature.
    - **Arguments**: Values passed to the function when it is called.

4. Default Parameter Values

- You can assign a default value to a parameter, making it optional.

```
def greet(name="Guest"):
    print(f"Hello, {name}")
```

5. Return Values

- Functions can return a value using the `return` statement.
- If no `return` statement is used, the function returns `None`.

Best Practices

1. Tips for Writing Efficient Python Code

- **Use Meaningful Variable Names**: Choose clear and descriptive names for variables and functions to make your code more readable and maintainable.
- **Avoid Repetition (DRY Principle)**: Follow the "Don't Repeat Yourself" principle by creating reusable functions and classes to avoid redundant code.
- **Use Built-in Functions**: Leverage Python's built-in functions and libraries (like `len()`, `max()`, `min()`, `sum()`, etc.) for optimized and clean code.
- **Write Modular Code**: Break down your code into smaller, reusable functions and classes, making it easier to debug and extend.
- **Handle Exceptions**: Always include error handling (`try-except` blocks) to manage exceptions and prevent program crashes.

2. Common Pitfalls

- **Mutable Default Arguments**: Avoid using mutable objects (e.g., lists, dictionaries) as default arguments in functions, as they can lead to unexpected behavior.
- **Indentation Errors**: Python relies on indentation for code blocks. Consistently use spaces (preferably 4 spaces per level) or tabs to avoid indentation errors.
- **Overuse of Global Variables**: Minimize the use of global variables as they can lead to code that is hard to maintain and debug.
- **Ignoring PEP 8**: Follow the Python Enhancement Proposal 8 (PEP 8) style guide for clean, readable, and consistent Python code.

3. Optimizing Code Performance

- **Use List Comprehensions**: Replace loops with list comprehensions for concise and faster code.
- **Use Generators**: Use generators for large datasets to handle memory efficiently.
- **Profiling**: Use tools like `timeit` or `cProfile` to measure the performance of your code and identify bottlenecks.
- **Efficient Data Structures**: Choose appropriate data structures (like sets, dictionaries) for optimal performance in data manipulation.

Additional Resources

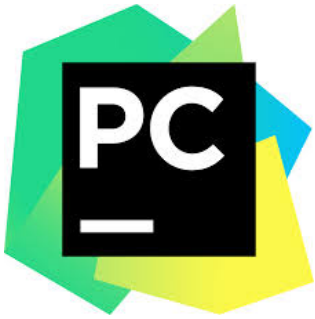
- https://www.py4e.com/lessons
- Learn Python Programming
- Reddit resources for python
- Python 101
- Python official documentation
- https://books.goalkicker.com/PythonBook/

Recommended Reading

- **"Python Crash Course" by Eric Matthes**: A comprehensive introduction to Python programming with hands-on projects.
- **"Fluent Python" by Luciano Ramalho**: An advanced guide focusing on writing idiomatic and effective Python code.
- **"Automate the Boring Stuff with Python" by Al Sweigart**: Perfect for beginners, this book teaches how to automate everyday tasks with Python.
- **"Effective Python: 90 Specific Ways to Write Better Python" by Brett Slatkin**: A collection of best practices and techniques for improving your Python code.

Python Tools and IDEs

- **PyCharm**: A powerful, full-featured IDE for Python development with intelligent code assistance and tools for debugging and testing.



- **Jupyter Notebook**: An interactive environment that allows you to write, test, and visualize Python code, making it ideal for data analysis and exploration.

- **Google Colab:** A free, cloud-based Jupyter notebook environment that provides access to GPU/TPU for running Python code, perfect for machine learning and data science projects.

- **Visual Studio Code (VS Code)**: A lightweight, highly customizable code editor with Python-specific extensions for syntax highlighting, linting, and debugging.

- **IPython**: An enhanced interactive Python shell with additional features like tab completion, syntax highlighting, and magic commands.

Conclusion

Mastering Python is key for software development, **machine learning, data analysis**, web development, and more.

This document provides an in-depth overview of Python fundamentals, OOP concepts, data structures, and advanced topics. By adhering to best practices and leveraging the recommended resources, you can enhance your Python skills, write efficient code, and become proficient in building robust applications.